Pabna University of Science and Technology

INFORMATION AND COMMUNICATION ENGINEERING

Course Name: Digital Communication Sessional

Course Code: ICE-3206

Submitted To,

Dr. Md. Imran Hossain
Associate Professor
Tarun DebNath
Assistant Professor
Department of ICE, PUST

Submitted By,

Md.MursaLin Lemon
Roll: 220606
Session:2021-22
3rd Year 2nd Semester
Department of ICE, PUST

INDEX

1. To develop a program and simulate the Pulse Amplitude Modulation (PAM) signal and analyze its waveform.

2. To design a program and simulate the Amplitude Shift Keying (ASK) modulation technique.

3. To construct a program and simulate the Phase Shift Keying (PSK) modulation technique.

4. To create a program and simulate the Frequency Shift Keying (FSK) modulation technique.

5. To implement a program and simulate the Unipolar Non-Return-to-Zero (NRZ) signaling technique.

6. To model a program and simulate the Polar Non-Return-to-Zero (NRZ) signaling technique.

7. To write a program and simulate the Unipolar Return-to-Zero (RZ) signaling technique.

8. To simulate a program for the Bipolar Return-to-Zero (RZ) signaling technique.

9. To develop a program and simulate the Manchester (Split-Phase) coding technique.

10. Perform the QPSK modulation and demodulation using MATLAB/ Python. Display the signal and its constellation.

11. Generate 16-QAM modulation and obtain the QAM constellation using MATLAB/ Python.

12. Perform sampling and reconstruction of continuous signal using MATLAB/ Python.

13 To generate the waveform of Quadrature Phase Shift Keying signal using MATLAB/ Python.

14. Perform the PAM modulation and demodulation using MATLAB/ Python.

15. Perform the DPCM modulation and demodulation using MATLAB/ Python.

Title : To develop a program and simulate the Pulse Amplitude Modulation (PAM) signal and analyze its waveform

Objectives :

- To understand the concept of Pulse Amplitude Modulation (PAM)
- To observe and analyze the waveform of the PAM signal
- To understand how sampling affects the signal

Theory :

Pulse amplitude modulation is defined as a process of varying the amplitude of the single pulse in accordance to the modulating signal variations. PAM is the basic form of analog pulse modulation in which the width and position characteristics of the pulse are kept constant while varying the amplitude. There are two sampling techniques used for sampling the modulating signal in PAM, which are Flat top sampling and Natural sampling. There are two types of PAM which are based on the criteria of signal polarity.

Natural PAM – Pulse follows actual signal shape

Flat-top PAM – Pulse amplitude is constant during pulse width (most common in practical systems)

Mathematical Representation

Let:

- Message signal:

$$m(t)=A\sin(2\pi f_m t)$$

- Sampling signal (pulse train):

$$p(t)=\sum\delta(t-nT_s)$$

- PAM signal:

$$s(t)=m(t)\cdot p(t)$$

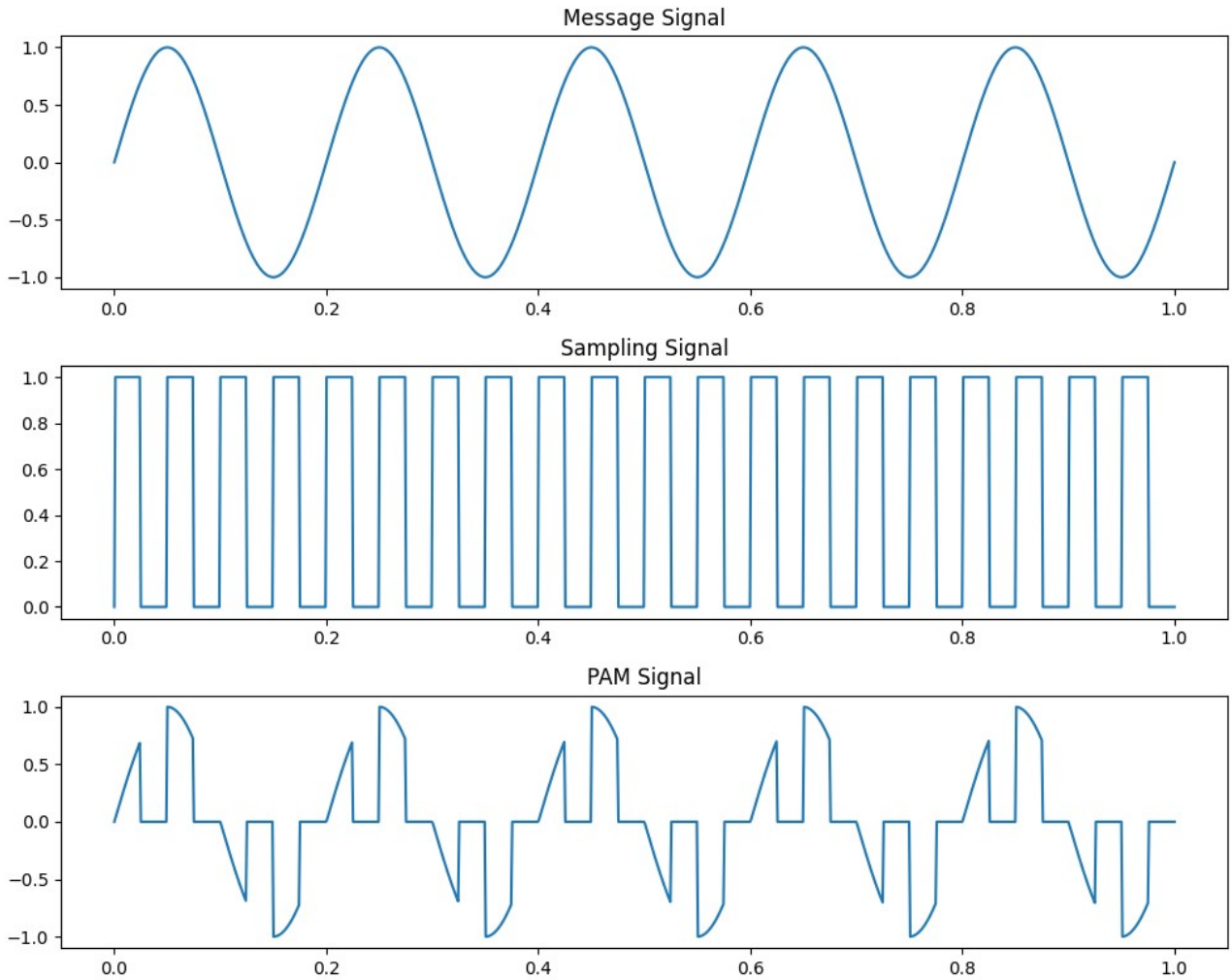
Here :

- We multiply the message signal with a **pulse train**
- This creates sampled pulses

source code :

```
import numpy as np

import matplotlib.pyplot as plt

t = np.linspace(0, 1, 1000)

fm = 5
Am = 1

message = Am * np.sin(2 * np.pi * fm * t)

fs = 20
sampling = np.sin(2 * np.pi * fs * t)
sampling = (sampling > 0).astype(int)

pam = message * sampling

plt.figure(figsize=(10,8))

plt.subplot(3,1,1)
plt.plot(t, message)
plt.title("Message Signal")

plt.subplot(3,1,2)
plt.plot(t, sampling)
plt.title("Sampling Signal")

plt.subplot(3,1,3)
plt.plot(t, pam)
plt.title("PAM Signal")

plt.tight_layout()
plt.show()
```

Output :



Result & Discussion :

- First graph: Continuous message signal (sine wave)
- Second graph: Pulse train (sampling signal)
- Third graph: PAM signal (sampled output)

The PAM signal shows pulses whose amplitudes follow the sine wave.

- PAM converts a continuous signal into a discrete-time signal
- It is the first step in digital communication systems
- Increasing sampling frequency improves signal accuracy
- If sampling frequency is too low → distortion (aliasing)
- PAM is used before PCM and Digital transmission systems

Title : To design a program and simulate the Amplitude Shift Keying (ASK) modulation technique.

Objectives :

- To understand the concept of Amplitude Shift Keying (ASK)
- To generate a digital binary signal
- To modulate the signal using ASK technique
- To observe and analyze the ASK waveform

Theory :

Amplitude Shift Keying (ASK) is a fundamental digital modulation technique used in communication systems to transmit binary data over a carrier signal. In ASK, the **amplitude of a high-frequency carrier wave is varied according to the digital input signal**, while the frequency and phase remain constant.

Basic Principle

In ASK modulation:

- A binary **'1'** is represented by transmitting a carrier signal with a certain amplitude.
- A binary **'0'** is represented by either:
  - No carrier signal (zero amplitude), or
  - A carrier with lower amplitude.

This means the carrier signal is **switched ON and OFF** based on the input digital data.

Mathematical Representation

Let:

- Carrier signal:

$$c(t)=A_c\cos(2\pi f_c t)$$

- Binary data:

$$m(t)=0 \text{ or } 1$$

- ASK signal:

$$s(t)=m(t)\cdot A_c\cos(2\pi f_c t)$$

here :

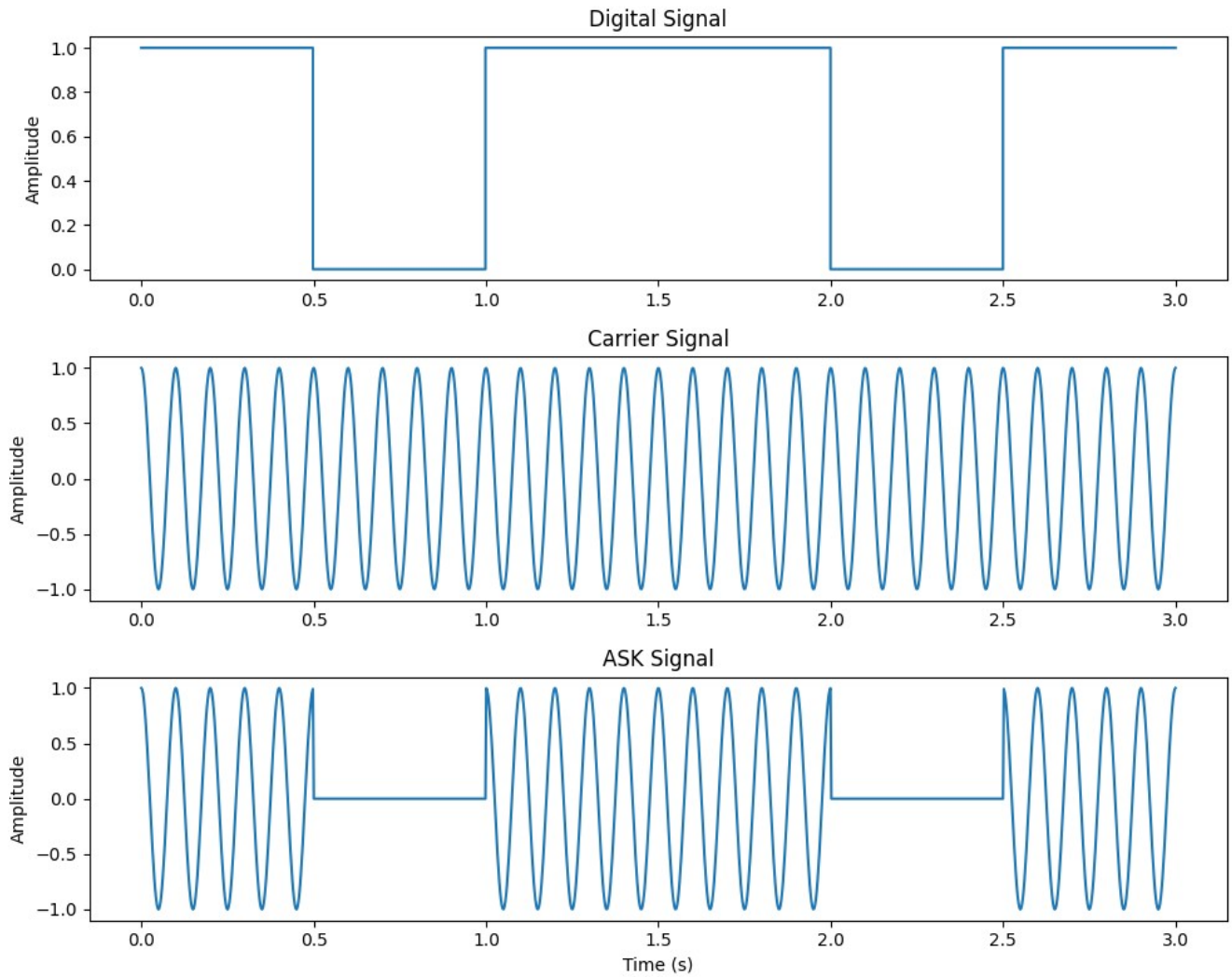
- If m(t)=1, signal is present
- If m(t)=0, signal is absent

source code :

```
import numpy as np

import matplotlib.pyplot as plt

bit_rate = 2

fc = 10

data = np.array([1, 0, 1, 1, 0, 1])

Tb = 1 / bit_rate

samples_per_bit = 200

t = np.linspace(0, Tb * len(data), samples_per_bit * len(data))

digital_signal = np.repeat(data, samples_per_bit)

carrier = np.cos(2 * np.pi * fc * t)

ask_signal = digital_signal * carrier

plt.figure(figsize=(10,8))

plt.subplot(3,1,1)

plt.step(t, digital_signal)

plt.title("Digital Signal")

plt.ylabel("Amplitude")

plt.subplot(3,1,2)

plt.plot(t, carrier)

plt.title("Carrier Signal")

plt.ylabel("Amplitude")
```

```
plt.subplot(3,1,3)
plt.plot(t, ask_signal)
plt.title("ASK Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.tight_layout()
plt.show()
```

Output :



Result & Discussion :

First graph: Digital input (binary sequence)

Second graph: Carrier signal
Third graph: ASK modulated signal

When bit = 1 → carrier is visible
When bit = 0 → signal disappears

ASK is one of the simplest digital modulation techniques

It is easy to implement but:

- Highly sensitive to noise

Used in:

- Optical communication
- RF remote controls

It requires:

- Synchronization between transmitter and receiver

Title : To construct a program and simulate the Phase Shift Keying (PSK) modulation technique.

Objectives :

- To understand the concept of Phase Shift Keying modulation.
- To generate BPSK modulated signal from binary data.
- To recover the original signal using demodulation.

Theory :

Phase Shift Keying (PSK) is a digital modulation technique in which the **phase of a carrier signal is varied according to the digital input data**, while the amplitude and frequency remain constant.

The simplest form of PSK is **Binary PSK (BPSK)**, where:

- Binary **1** is represented by a carrier with **0° phase**
- Binary **0** is represented by a carrier with **180° phase shift**

The BPSK signal can be expressed as:

$$s(t)=A\cos(2\pi f_c t+\theta)$$

Where:

- A = amplitude of the carrier
- f_c = carrier frequency
- $\theta=0$ for bit 1
- $\theta=\pi$ for bit 0

This can also be written as:

$$s(t)=A\cdot m(t)\cdot\cos(2\pi f_c t)$$

Where:

- $m(t)=+1$ for bit 1
- $m(t)=-1$ for bit 0

In PSK modulation:

1. The input binary data is first converted into **polar form** (+1 and −1).

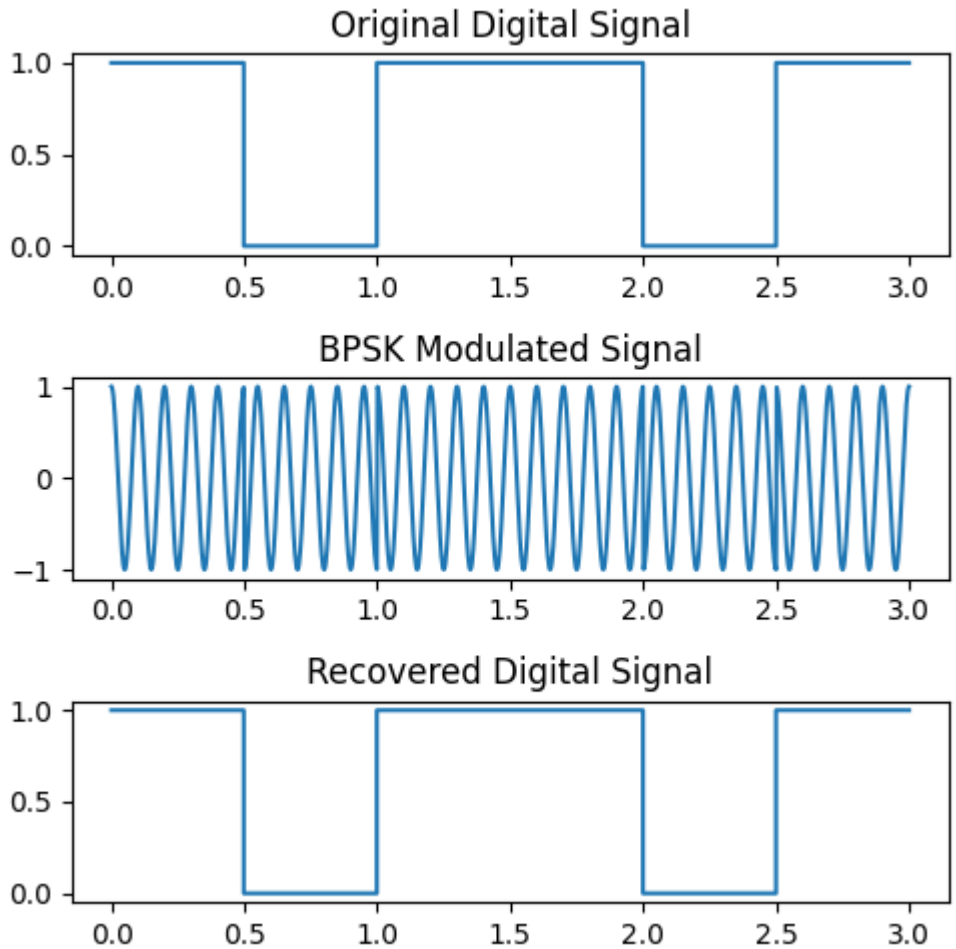
2. This polar signal is multiplied with the carrier signal.
3. The output is a **phase-shifted waveform**:
    - No phase change → bit 1
    - 180° phase shift → bit 0

source code :

```
import numpy as np

import matplotlib.pyplot as plt

data = np.array([1, 0, 1, 1, 0, 1])

bit_rate, fc = 2, 10

Tb = 1 / bit_rate

samples_per_bit = 200

t = np.linspace(0, Tb * len(data), samples_per_bit * len(data))

digital_signal = np.repeat(data, samples_per_bit)

carrier = np.cos(2 * np.pi * fc * t)

polar_data = 2 * digital_signal – 1 ; psk_signal = polar_data * carrier

demodulated = psk_signal * carrier ; recovered_bits = []

demod_signal_smooth = np.zeros_like(demodulated)

for i in range(len(data)):

    start = i * samples_per_bit

    end = (i + 1) * samples_per_bit

    avg = np.mean(segment)

recovered_signal = np.repeat(recovered_bits, samples_per_bit)

plt.figure(figsize=(10,10))

plt.subplot(3,1,1)

plt.step(t, digital_signal)

plt.title("Original Digital Signal")
```

```
plt.subplot(3,1,2)
plt.plot(t, psk_signal)
plt.title("BPSK Modulated Signal")
plt.subplot(3,1,3)
plt.step(t, recovered_signal)
plt.title("Recovered Digital Signal")
plt.tight_layout()
plt.show()
```

Output :



Original bits: [1 0 1 1 0 1]

Recovered bits: [1 0 1 1 0 1]

Result & Discussion :

The **Binary Phase Shift Keying (BPSK)** modulation and demodulation were successfully simulated using Python.

- The input binary sequence was correctly converted into a **phase-shifted carrier signal**.
- During modulation:
  - Bit **1** was represented by a carrier with **0° phase**
  - Bit **0** was represented by a **180° phase shift**
- The demodulation process (coherent detection + averaging) successfully recovered the original data.
- The **recovered digital signal matched the transmitted signal**.

The simulation demonstrates the working principle of **BPSK modulation**, where only the **phase of the carrier signal changes** while amplitude and frequency remain constant.

- The modulated waveform clearly shows **phase reversals** corresponding to binary data.
- In the demodulation process:
  - Multiplying with a synchronized carrier converts the signal back to baseband.
  - Averaging (integration) helps remove high-frequency components.
  - A threshold decision accurately reconstructs the original bits.

Title : To create a program and simulate the Frequency Shift Keying (FSK) modulation technique.

Objectives :

- To understand **Frequency Shift Keying (FSK)**
- To generate binary digital data
- To simulate FSK modulation

Theory :

Frequency Shift Keying (FSK) is a digital modulation technique where:

The **frequency of the carrier signal is varied according to the binary data**

In simple words:

- Binary **1 → High frequency (f_1)**
- Binary **0 → Low frequency (f_o)**

Basic Concept

- Input → Digital signal (0s and 1s)
- Output → Signal with **two different frequencies**

For binary signals:

$$s(t)=\begin{cases} A_c\cos(2\pi f_1 t), & \text{if bit = 1} \\ A_c\cos(2\pi f_0 t), & \text{if bit = 0} \end{cases}$$

Where:

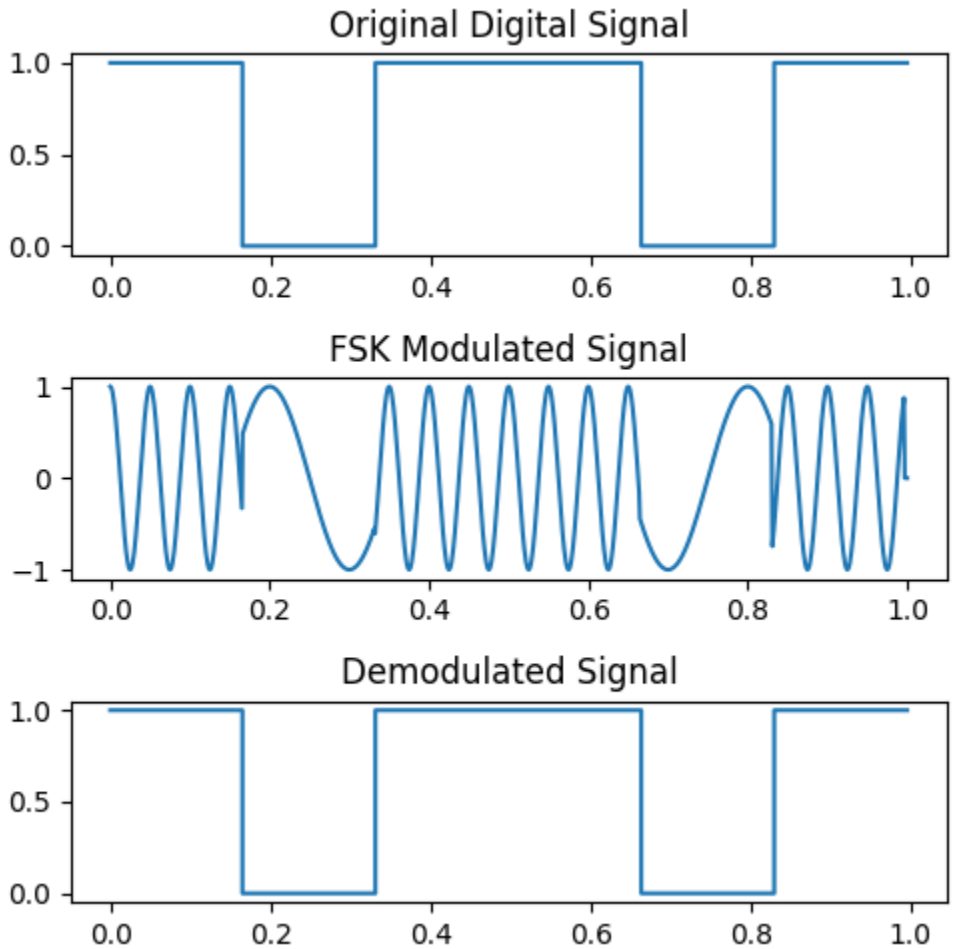
- f1 = frequency for bit 1
- f0 = frequency for bit 0

source code :

```
import numpy as np
import matplotlib.pyplot as plt
bit_rate = 2
t_total = 1
```

```
t = np.linspace(0, t_total, 1000)
f1 = 20 # for bit 1
f0 = 5 # for bit 0
data = np.array([1, 0, 1, 1, 0, 1])
bit_samples = len(t) // len(data)
fsk_signal = np.zeros_like(t)
for i, bit in enumerate(data):
start = i * bit_samples
end = start + bit_samples
if bit == 1:
fsk_signal[start:end] = np.cos(2 * np.pi * f1 * t[start:end])
else:
fsk_signal[start:end] = np.cos(2 * np.pi * f0 * t[start:end])
demodulated = []

for i in range(len(data)):
start = i * bit_samples
end = start + bit_samples

segment = fsk_signal[start:end]
ref1 = np.cos(2 * np.pi * f1 * t[start:end])
ref0 = np.cos(2 * np.pi * f0 * t[start:end])
corr1 = np.sum(segment * ref1)
corr0 = np.sum(segment * ref0)
if corr1 > corr0:
demodulated.append(1)
else:
demodulated.append(0)
demod_signal = np.repeat(demodulated, bit_samples)
plt.figure(figsize=(5,5))
plt.subplot(3,1,1)
plt.step(t[:len(np.repeat(data, bit_samples))], np.repeat(data, bit_samples))
plt.title("Original Digital Signal")
plt.subplot(3,1,2)
plt.plot(t, fsk_signal)
plt.title("FSK Modulated Signal")
plt.subplot(3,1,3)
plt.step(t[:len(demod_signal)], demod_signal)
plt.title("Demodulated Signal")
plt.tight_layout()
plt.show()
print("Original Data: ", data)
print("Demodulated Data: ", demodulated)
```

Output :



Original bits: [1 0 1 1 0 1]

Recovered bits: [1 0 1 1 0 1]

Result & Discussion :

- 1st graph → **Original binary signal**
- 2nd graph → **FSK modulated signal**
- 3rd graph → **Recovered (demodulated) signal**

Different frequencies for 0 and 1 .Demodulated signal matches original data . FSK changes **frequency**, not amplitude or phase . More robust than ASK because:
Noise affects amplitude more than frequency .Demodulation works by: Detecting which frequency is stronger

Title

To implement a program and simulate the Unipolar Non-Return-to-Zero (NRZ) signaling technique.

Objectives

- To understand the concept of Unipolar NRZ signaling.
- To generate NRZ waveform from binary data.
- To analyze the time-domain representation of digital signals.

Theory

Unipolar Non-Return-to-Zero (NRZ) is a line coding technique in which binary data is represented using a single polarity. In this scheme, a binary '1' is represented by a positive voltage level, while a binary '0' is represented by zero voltage.

The NRZ signal can be expressed as:

$$s(t) = \begin{cases} A, & \text{for bit 1} \\ 0, & \text{for bit 0} \end{cases} \tag{1}$$

where A is the amplitude of the signal.

In NRZ signaling, the signal level remains constant throughout the bit duration and does not return to zero between consecutive bits, which improves bandwidth efficiency.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.random.randint(0, 2, N)

samples_per_bit = 100
y = np.repeat(n, samples_per_bit)

t = np.linspace(0, N, len(y))

plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
plt.ylim(-1.5, 1.5)
plt.title("Unipolar NRZ Signaling")
plt.xlabel("Time")
plt.ylabel("Amplitude")
```

```
plt.grid()

for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)

plt.show()
```

Output

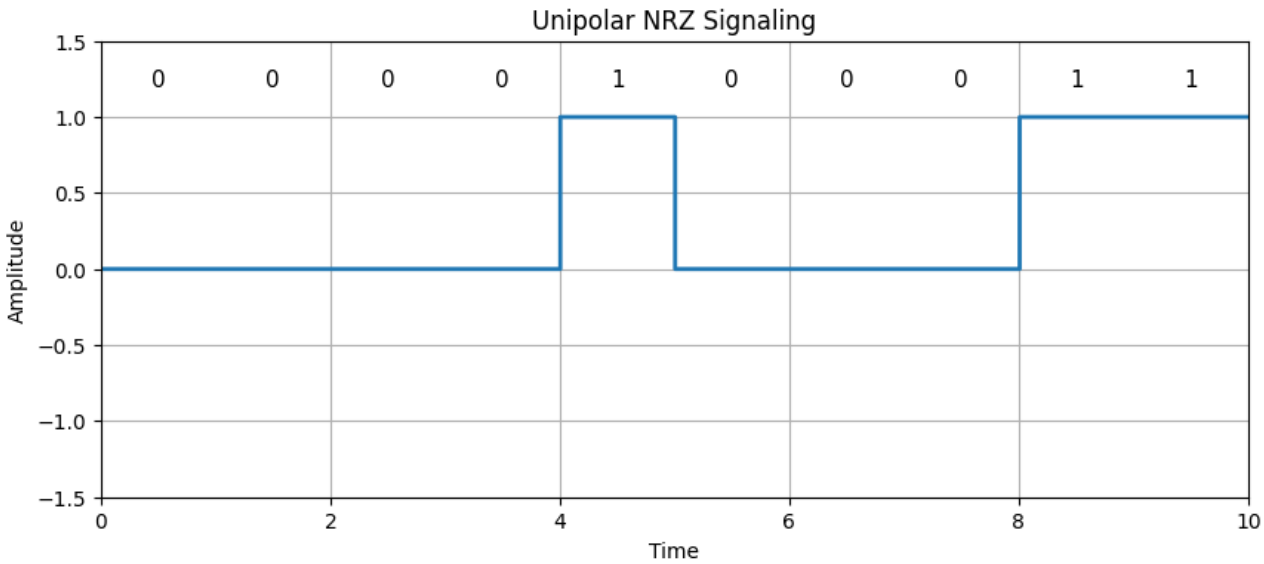


Figure : Unipolar NRZ Signaling Waveform

Results and Discussions

The NRZ signal is generated directly from binary input data. Binary 1 is represented by a high level and 0 by zero level. The signal remains constant during each bit interval. No return to zero occurs between consecutive bits. Thus, Unipolar NRZ signaling is successfully implemented.

Title

To model a program and simulate the Polar Non-Return-to-Zero (NRZ) signaling technique.

Objectives

- To understand the concept of Polar NRZ signaling.
- To generate Polar NRZ waveform from binary data.
- To analyze signal representation using positive and negative levels.

Theory

Polar Non-Return-to-Zero (NRZ) is a line coding technique in which binary data is represented using two voltage levels of opposite polarity. A binary '1' is represented by a positive voltage, while a binary '0' is represented by a negative voltage. This approach improves signal balance and reduces DC components compared to unipolar signaling.

The Polar NRZ signal can be expressed as:

$$s(t) = \begin{cases} +A, & \text{for bit 1} \\ -A, & \text{for bit 0} \end{cases} \tag{1}$$

where A is the signal amplitude. The signal level remains constant throughout the bit duration and does not return to zero between successive bits. The use of bipolar levels helps in better synchronization and improves noise immunity.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.random.randint(0, 2, N)

nn = np.where(n == 1, 1, -1)

samples_per_bit = 100
y = np.repeat(nn, samples_per_bit)

t = np.linspace(0, N, len(y))

plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
plt.ylim(-1.5, 1.5)
plt.title("Polar NRZ Signaling")
```

```
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()

for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)

plt.show()
```

Output

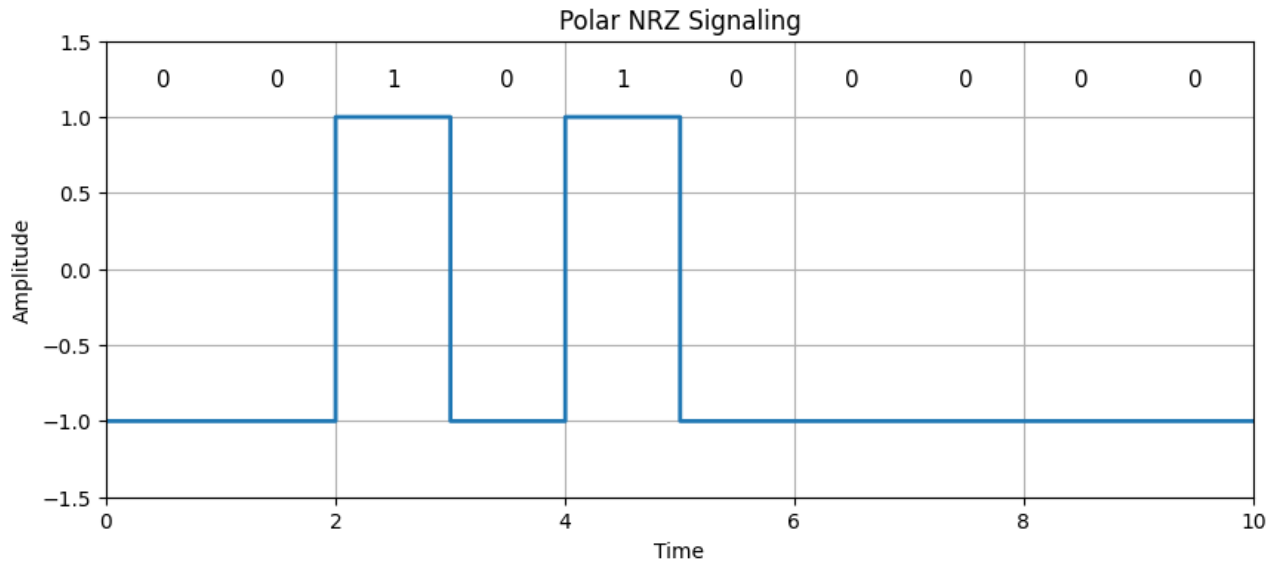


Figure : Polar NRZ Signaling Waveform

Results and Discussions

The generated waveform clearly shows two distinct levels for binary data. Positive amplitude represents bit 1 while negative amplitude represents bit 0. The signal remains constant over each bit duration without returning to zero. Use of bipolar levels improves signal balance and reduces DC component. Thus, Polar NRZ provides better performance compared to unipolar signaling.

Title

To write a program and simulate the Unipolar Return-to-Zero (RZ) signaling technique.

Objectives

- To understand the principle of Unipolar RZ signaling.
- To generate RZ waveform from binary data.
- To analyze signal characteristics and transitions.

Theory

Unipolar Return-to-Zero (RZ) is a line coding technique in which binary data is represented using a single polarity, and the signal returns to zero within each bit duration. A binary '1' is represented by a positive pulse for half of the bit period followed by zero, while a binary '0' remains at zero throughout the bit duration.

The RZ signal can be expressed as:

$$s(t) = \begin{cases} A, & 0 \leq t < \frac{T}{2} \quad \text{for bit 1} \\ 0, & \frac{T}{2} \leq t < T \\ 0, & 0 \leq t < T \quad \text{for bit 0} \end{cases} \tag{1}$$

where A is the amplitude and T is the bit duration. The return-to-zero characteristic ensures better synchronization but increases bandwidth requirements compared to NRZ signaling.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10; n = np.random.randint(0, 2, N)
samples_per_bit = 100
half = samples_per_bit // 2
y = []
for bit in n:
    if bit == 1:
        y.extend([1]*half + [0]*half)
    else:
        y.extend([0]*samples_per_bit)

y = np.array(y)
t = np.linspace(0, N, len(y))
plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
```

```
plt.ylim(-1.5, 1.5)
plt.title("Unipolar Return-to-Zero (RZ) Signaling")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)
plt.show()
```

Output

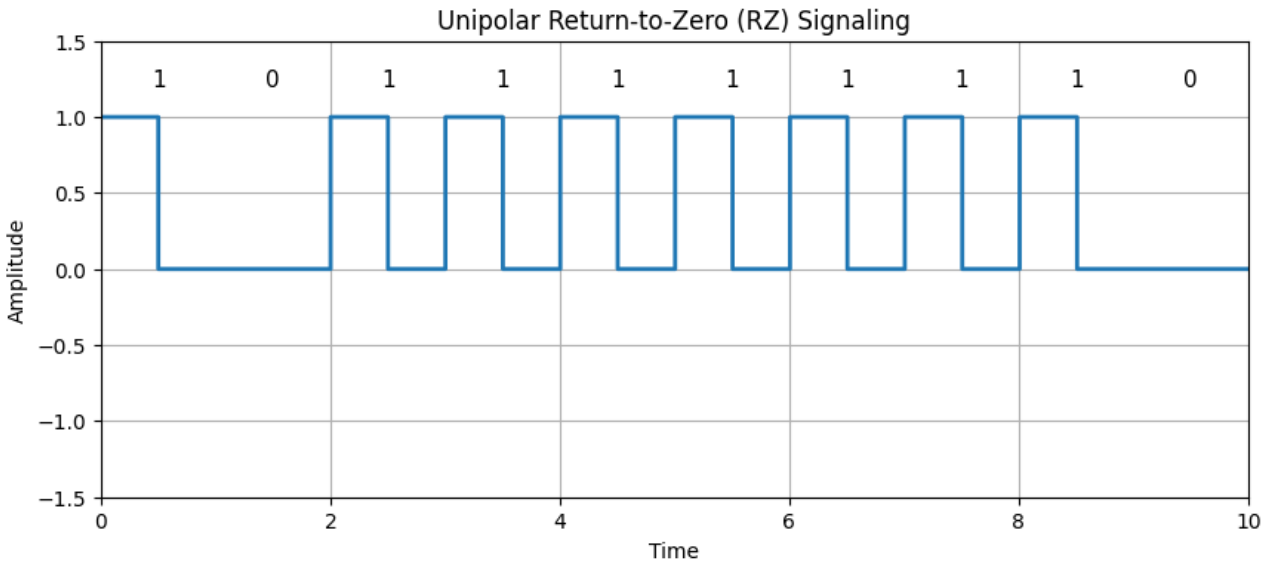


Figure : Unipolar Return-to-Zero (RZ) Signaling Waveform

Results and Discussions

The waveform shows pulses that return to zero within each bit interval. Binary 1 is represented by a half-duration pulse followed by zero level. Binary 0 remains at zero throughout the bit duration. Frequent transitions improve synchronization at the receiver. However, increased transitions require higher bandwidth compared to NRZ.

Title

To simulate a program for the Bipolar Return-to-Zero (RZ) signaling technique.

Objectives

- To understand Bipolar RZ (AMI) signaling technique.
- To generate waveform with alternating polarity pulses.
- To analyze signal characteristics and DC balance.

Theory

Bipolar Return-to-Zero (RZ), also known as Alternate Mark Inversion (AMI), is a line coding technique in which binary '1's are represented by alternating positive and negative voltage pulses, while binary '0's are represented by zero voltage. In addition, the signal returns to zero within each bit duration.

The Bipolar RZ signal can be expressed as:

$$s(t) = \begin{cases} +A, & \text{for bit 1 (positive pulse)} \\ -A, & \text{for bit 1 (negative pulse)} \\ 0, & \text{for bit 0} \end{cases} \tag{1}$$

The polarity of successive '1's alternates to maintain a zero DC component. The return-to-zero property ensures that each pulse occupies only a portion of the bit duration, improving synchronization.

This technique combines the advantages of bipolar signaling and RZ format, making it suitable for reliable data transmission.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10; n = np.random.randint(0, 2, N)
nn = np.zeros(N)
polarity = 1
for i in range(N):
    if n[i] == 1:
        nn[i] = polarity
        polarity *= -1
    else:
        nn[i] = 0
samples_per_bit = 100
half = samples_per_bit // 2

y = []
```

```
for val in nn:
    if val != 0:
        y.extend([val]*half + [0]*half)
    else:
        y.extend([0]*samples_per_bit)

y = np.array(y)
t = np.linspace(0, N, len(y))
plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
plt.ylim(-1.5, 1.5)
plt.title("Bipolar Return-to-Zero (BRZ) Signaling")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)
plt.show()
```

Output

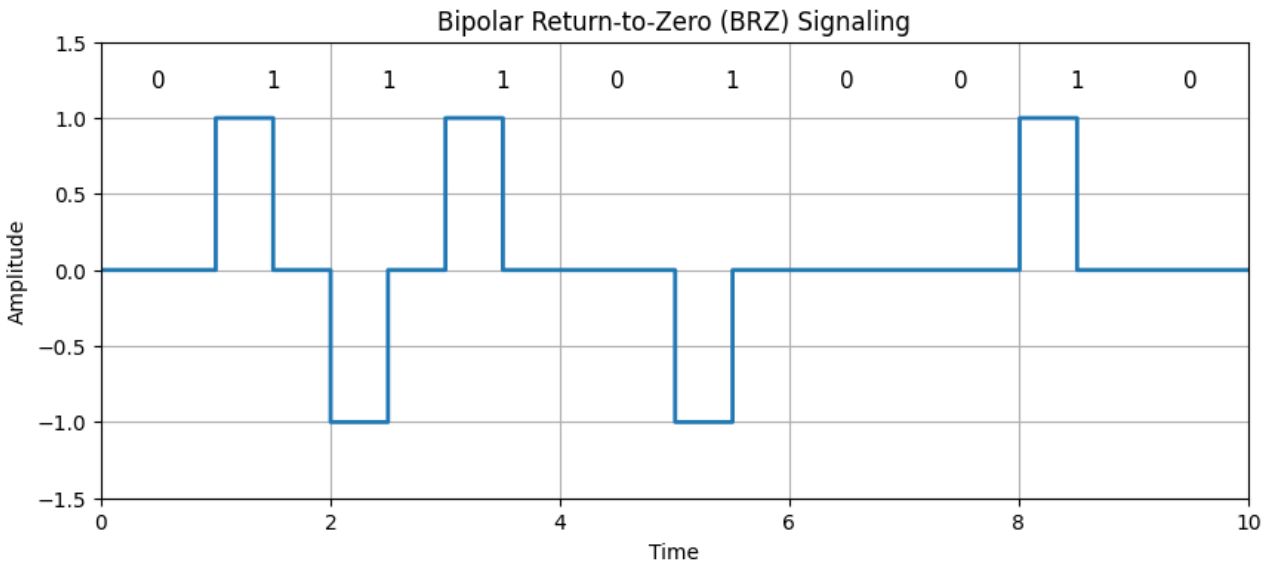


Figure 1 : Bipolar Return-to-Zero (RZ) Signaling Waveform

Results and Discussions

The waveform shows alternating positive and negative pulses for binary 1. Binary 0 is represented by zero amplitude throughout the bit duration. The signal returns to zero within each bit interval. Alternating polarity ensures zero DC component and better signal balance. Thus, Bipolar RZ provides improved synchronization and transmission performance.

Title

To develop a program and simulate the Manchester (Split-Phase) coding technique.

Objectives

- To understand the principle of Manchester encoding.
- To generate Manchester coded waveform from binary data.
- To analyze synchronization and signal transitions.

Theory

Manchester coding, also known as split-phase encoding, is a line coding technique in which each bit is represented by a transition in the middle of the bit duration. This ensures that the signal contains regular transitions, which helps in clock synchronization at the receiver.

In Manchester encoding, a binary '1' is represented by a high-to-low transition, while a binary '0' is represented by a low-to-high transition within each bit period.

The Manchester signal can be expressed as:

$$s(t) = \begin{cases} +A \to -A, & \text{for bit 1} \\ -A \to +A, & \text{for bit 0} \end{cases} \tag{1}$$

where A is the signal amplitude. Since each bit contains a transition, the average DC component is zero, and timing information is embedded within the signal itself.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.random.randint(0, 2, N)

encoded = []

for bit in n:
    if bit == 1:
        encoded.extend([1, -1])
    else:
        encoded.extend([-1, 1])

samples_per_half_bit = 50
y = np.repeat(encoded, samples_per_half_bit)

t = np.linspace(0, N, len(y))
```

```
plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
plt.ylim(-1.5, 1.5)
plt.title("Manchester Coding")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()

for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)

plt.show()
```

Output

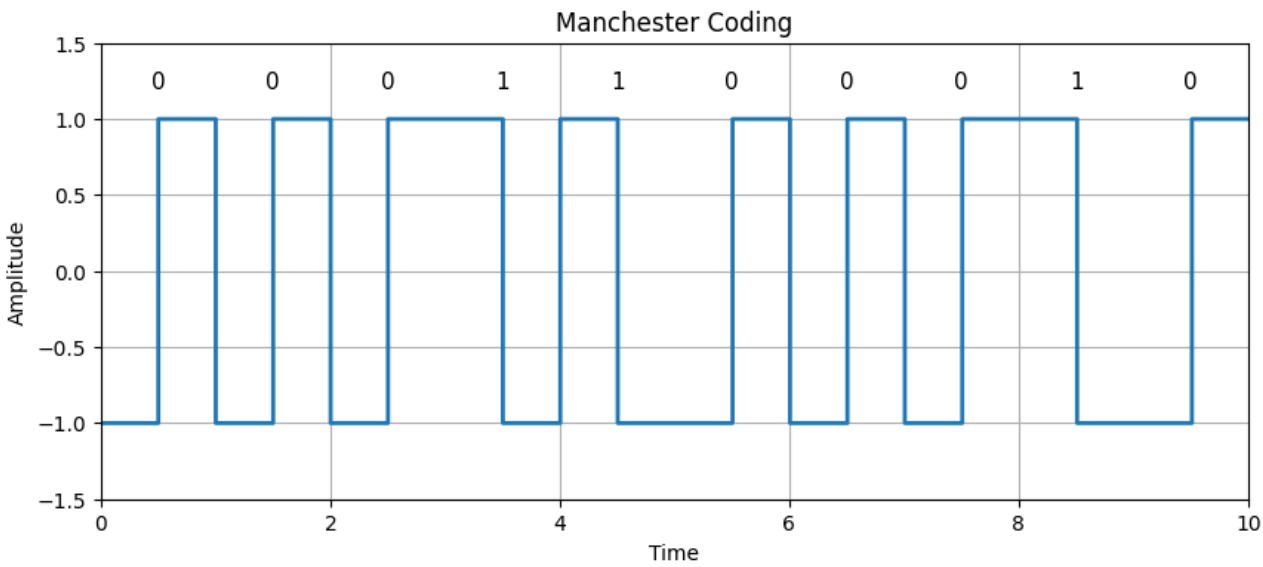


Figure 15: Manchester Coding Waveform

Results and Discussions

The waveform shows a transition in the middle of every bit period. Binary 1 produces a high-to-low transition, while 0 produces a low-to-high transition. The presence of transitions ensures reliable clock synchronization. The signal has no DC component due to equal positive and negative levels. Thus, Manchester coding provides efficient and reliable data transmission.

Title

Perform the QPSK modulation and demodulation using MATLAB/ Python. Display the signal and its constellation.

Objectives

- To understand the working principle of QPSK modulation.
- To implement QPSK modulation and demodulation using Python.
- To analyze the waveform and constellation diagram.

Theory

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique where two bits are transmitted simultaneously by varying the phase of a carrier signal. This increases bandwidth efficiency compared to binary modulation schemes.

In QPSK, binary data is grouped into pairs and each pair is mapped into a symbol represented by in-phase (I) and quadrature (Q) components. These components modulate cosine and sine carrier signals respectively.

The QPSK signal is expressed as:

$$s(t) = I(t)\cos(2\pi f_c t) + Q(t)\sin(2\pi f_c t)$$

where $I(t)$ and $Q(t)$ represent the in-phase and quadrature components and f_c is the carrier frequency.

The symbol mapping is defined as:

$$(0, 0) \to (1, 1), \quad (0, 1) \to (-1, 1), \quad (1, 1) \to (-1, -1), \quad (1, 0) \to (1, -1)$$

At the receiver, demodulation is performed by detecting the sign of the I and Q components to recover the original binary data.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.randint(0, 2,
20) pairs = data.reshape(-1, 2)

map_dict = {(0,0):(1,1), (0,1):(-1,1), (1,1):(-1,-1), (1,0):(1,-1)}
symbols = np.array([map_dict[tuple(p)] for p in pairs

]) I, Q = symbols[:,0], symbols[:,1]

t = np.linspace(0, 1, 100)
```

```
fc = 5
signal = np.concatenate([
    I[i]* np.cos(2* np.pi* fc* t) + Q[i]* np.sin(2* np.pi* fc*
    t) for i in range(len(I))
])

demod = []
for i, q in zip(I, Q):
    if i>0 and q>0: demod += [0,0]
    elif i<0 and q>0: demod += [0,1]
    elif i<0 and q<0: demod += [1,1]
    else: demod += [1,0]

demod = np.array(demod)

plt.figure(figsize=(12
,8)) plt.subplot(2,2,1)
plt.step(range(len(data)), data, where='
mid') plt.title("Original Binary
Signal") plt. xlabel(" Bit
Index") plt.ylabel("Amplitude")
plt.ylim(-0.5, 1.5)
plt.grid()

plt.subplot(2,2,2)
plt.plot(signal)
plt.title("QPSK Modulated
Signal") plt. xlabel(" Time
") plt.ylabel("Amplitude")
plt.grid()

plt.subplot(2,2,3)
plt.step(range(len(demod)), demod, where='mid')
plt.title("Demodulated Signal")
plt.xlabel("Bit Index")
plt.ylabel("Amplitude")
plt.ylim(-0.5, 1.5)
plt.grid()

plt.subplot (2 ,2 ,4)
plt.scatter(I,Q)
plt.title("QPSK Constellation")
plt.xlabel("In-phase (I)")
plt.ylabel("Quadrature (Q)")
plt.axhline(0)
plt.axvline(0)
plt.grid()
plt.tight_layout()
print("Original Data:", data)
print("Demodulated Data:", demod)
plt.show()
```

Output

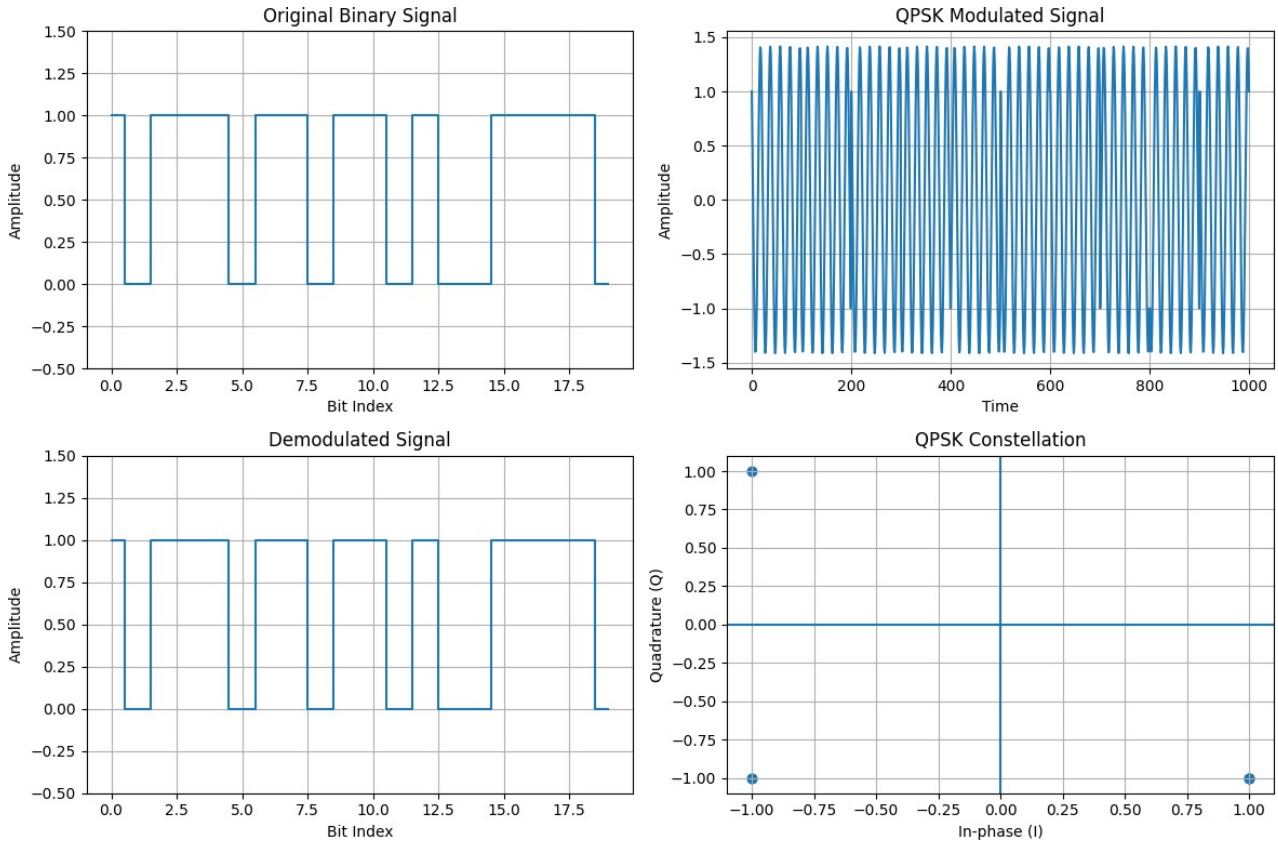


Figure : QPSK Modulated Signal, Demodulated Signal and Constellation Diagram

Results and Discussions

The QPSK modulation converts binary data into phase-shifted signals efficiently. The demodulated output closely matches the original input sequence. The constellation diagram shows four clearly separated symbol points. This indicates proper mapping and accurate detection of symbols. Thus, the system demonstrates correct modulation and demodulation.

Title

Generate 16-QAM modulation and obtain the QAM constellation using MATLAB/ Python.

Objectives

- To understand the working principle of 16-QAM modulation.
- To generate modulated QAM signals using Python.
- To analyze the waveform and constellation diagram.

Theory

Quadrature Amplitude Modulation (QAM) is a digital modulation technique that combines both amplitude and phase variations of a carrier signal. In 16-QAM, each symbol represents 4 bits, which increases the data transmission rate.

The QAM signal is expressed as:

$$s(t) = I(t)\cos(2\pi f_c t) + Q(t)\sin(2\pi f_c t)$$

where $I(t)$ and $Q(t)$ are the in-phase and quadrature components.

In 16-QAM, the amplitude levels are defined as:

$$I, Q \in \{-3, -1, +1, +3\}$$

Gray coding is used to map bits into symbols to minimize the probability of bit errors.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 200
bits = np.random.randint(0, 2, N
*4) bit_groups = bits.reshape((N,
4))

    (0,0): -3,
    (0,1): -1,
    (1,1):  1,
    (1,0):  3
}

I = np.array([mapping[tuple(b[:2])] for b in bit_groups
]) Q = np.array([mapping[tuple(b[2:])] for b in
bit_groups])

qam_signal = I + 1j * Q
```

```
plt. figure ( figsize =(12

,5)) plt. subplot (1 ,2 ,1)
plt. plot(t, np. real( qam_signal), label='In - phase ( I)')
plt. plot(t, np. imag( qam_signal), label=' Quadrature (
Q)') plt. title (" 16 - QAM Modulated Signal")
plt. xlabel(" Symbol
Index") plt. ylabel("
Amplitude ") plt. legend ()
plt. grid ()

plt. subplot (1 ,2 ,2)
plt. scatter( np. real( qam_signal), np. imag(
qam_signal)) plt. title (" 16 - QAM Constellation
Diagram ") plt. xlabel(" In - phase ( I)")
plt. ylabel(" Quadrature ( Q)")
plt. xticks ([-3 , -1 , 1 , 3])
plt. yticks ([-3 , -1 , 1 ,
3]) plt. grid ()
plt. axis(' equal ')

plt. tight_layout ()
plt. show ()
```

Output

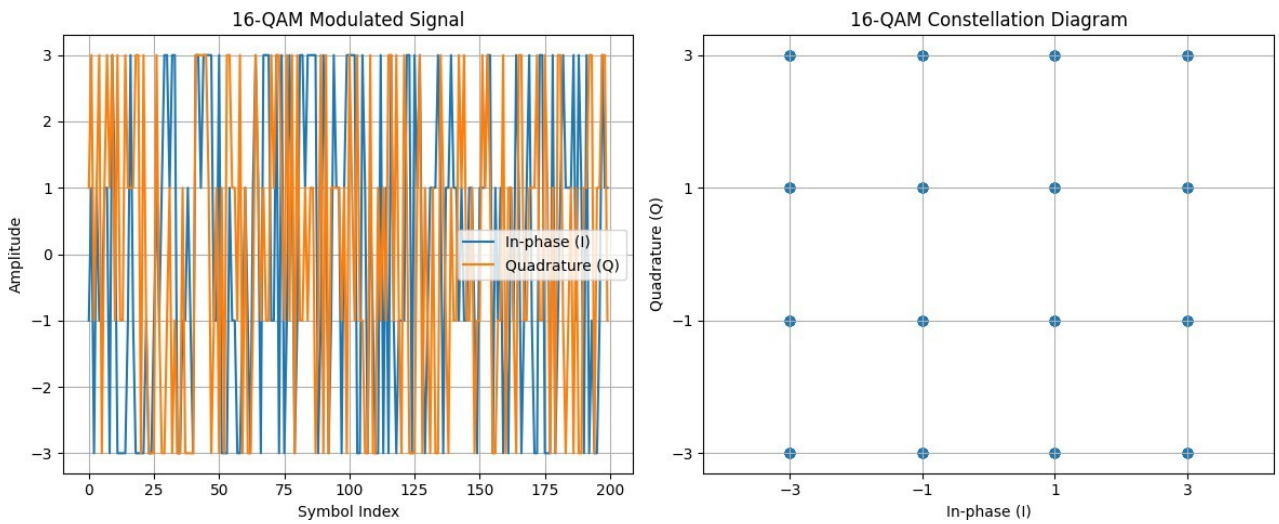


Figure : 16-QAM Modulated Signal and Constellation Diagram

Results and Discussions

The 16-QAM modulation generates signals using multiple amplitude levels. Both in-phase and quadrature components vary with symbol index. The constellation diagram shows 16 distinct symbol points. Each point represents a unique 4-bit combination. Thus, the system efficiently increases data transmission rate.

Title

Perform sampling and reconstruction of continuous signal using MATLAB/ Python.

Objectives

- To understand the concept of signal sampling.
- To perform sampling of a continuous-time signal.
- To reconstruct the original signal from sampled data.

Theory

Sampling is the process of converting a continuous-time signal into a discrete-time signal by taking measurements at regular time intervals. According to the Nyquist sampling theorem, the sampling frequency must be at least twice the highest frequency of the signal to avoid aliasing.

A continuous-time sinusoidal signal is represented as:

$$x(t) = \sin(2\pi f t) \tag{1}$$

The sampled signal is obtained as:

$$x[n] = x(nT_s) \tag{2}$$

where $T_s = \frac{1}{f_s}$ is the sampling interval and f_s is the sampling frequency.

The reconstruction of the signal is performed using interpolation techniques to approximate the original continuous signal.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1
d

t = np.linspace(0, 1, 1000)
f = 5

fs = 20
ts = np.arange(0, 1, 1/fs)
xs = np.sin(2 * np.pi * f *
     ts)
interp_func = interp1d(ts, xs, kind='linear
')
t_new = np.linspace(ts.min(), ts.max(),
1000) x_recon = interp_func(t_new)
```

```
plt. subplot (3 ,1 ,1)
plt. plot(t, x)
plt. title (" Original Continuous
Signal") plt.   xlabel("   Time              (
s)") plt. ylabel(" Amplitude ")
plt. grid ()

plt.   subplot   (3   ,1   ,2)
plt. stem ( ts , xs , basefmt="
") plt. title (" Sampled
Signal") plt. xlabel(" Time
( s)") plt. ylabel(" Amplitude
") plt. grid ()

plt. subplot (3 ,1 ,3)
plt.     plot(t,     x,     label="  Signal")
Original   plt.   plot(   t_new   ,  label=" Reconstructed   Signal")
x_recon , '--', plt. scatter( ts
plt. title (" Signal   Reconstruction
")
plt. xlabel(" Time ( s)")
plt. ylabel(" Amplitude ")
plt. legend ()
plt. grid ()

plt. tight_layout ()
```

Output

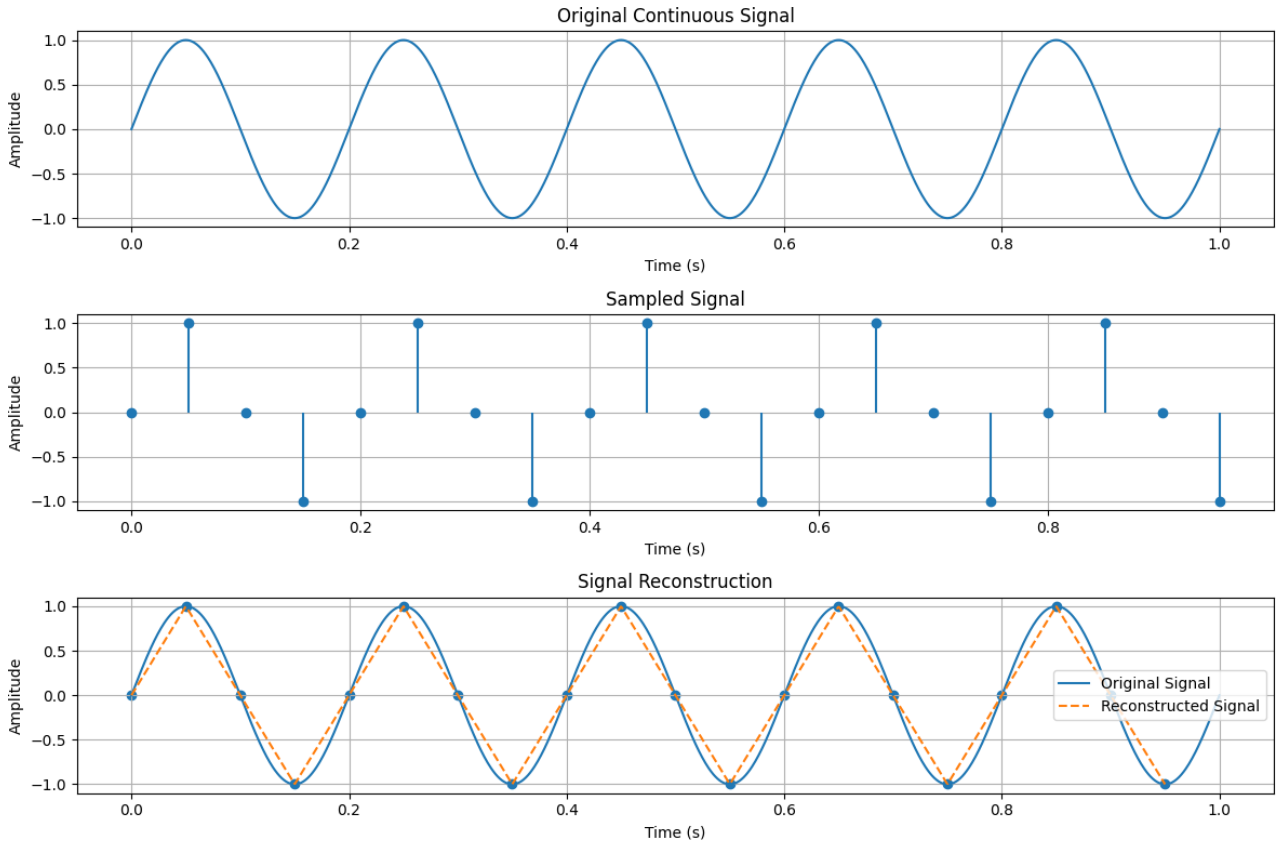


Figure : Sampling and Reconstruction of Continuous Signal

Results and Discussions

The continuous signal is successfully sampled at a fixed sampling frequency. The sampled signal represents discrete-time values of the original signal. The reconstructed signal closely follows the original waveform. Minor differences occur due to interpolation method used. Thus, proper sampling enables accurate signal reconstruction.

Title

To generate the waveform of Quadrature Phase Shift Keying signal using MATLAB/ Python.

Objectives

- To understand the generation of QPSK waveform.
- To analyze in-phase and quadrature components.
- To observe the final QPSK modulated signal.

Theory

Quadrature Phase Shift Keying (QPSK) is a modulation technique in which two bits are transmitted per symbol by changing the phase of the carrier signal. The signal is formed using two orthogonal carriers: cosine (in-phase) and sine (quadrature).

The QPSK signal is given by:

$$s(t) = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t)$$

where $I(t)$ and $Q(t)$ represent the in-phase and quadrature components.

The binary input is grouped into pairs and mapped into corresponding I and Q values using Gray coding. These components are then multiplied with carrier signals to generate the final QPSK waveform.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt
num_bits = 20; fc = 5; samples_per_bit = 100

bits = np.random.randint(0, 2, num_bits)
bit_pairs = bits.reshape(-1, 2)
mapping = {
    (0,0):(1,1), (0,1):(-1,1),
    (1,1):(-1,-1), (1,0):(1,-1)
}
I, Q = [], []
for pair in bit_pairs:
    i, q = mapping[tuple(
    pair)] I += [i]*
    samples_per_bit
    Q += [q]* samples_per_bit
I, Q = np.array(I), np.array(Q)
t = np.arange(len(I)) / samples_per_bit
```

```
carrier_Q = np. sin (2* np. pi* fc* t)
qpsk_signal = I* carrier_I - Q* carrier_Q

plt. figure ( figsize =(10
,6)) plt. subplot (3 ,1 ,1)
plt. plot(t, I)
plt. title (" In - phase(I)")
plt. subplot (3 ,1 ,2)
plt. plot(t, Q)
plt. title (" Quadrature (Q)")
plt. subplot (3 ,1 ,3)
plt. plot(t, qpsk_signal)
plt. title (" QPSK
Signal") plt.
tight_layout ()
plt. show ()
```

Output

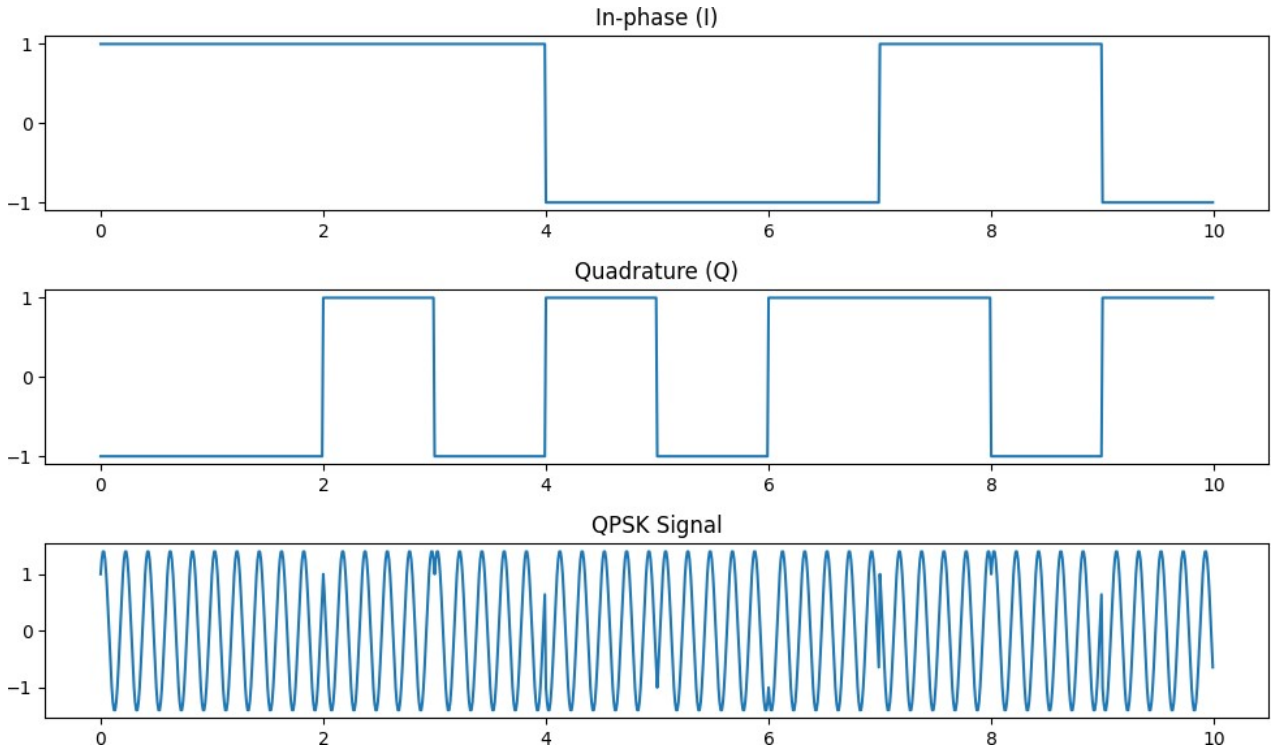


Figure : In-phase, Quadrature and QPSK Waveform

Results and Discussions

The in-phase and quadrature signals are generated correctly from bit pairs. These components vary according to the input binary data. The final QPSK signal is formed by combining both carriers. The waveform shows phase variations corresponding to symbols. Thus, the QPSK signal generation is successfully verified.

Title

Perform the PAM modulation and demodulation using MATLAB/ Python.

Objectives

- To understand the concept of Pulse Amplitude Modulation (PAM).
- To implement PAM modulation and demodulation.
- To analyze the effect of noise and signal recovery.

Theory

Pulse Amplitude Modulation (PAM) is a technique where the amplitude of equally spaced pulses is varied according to the message signal. In digital PAM, binary data is mapped into discrete amplitude levels.

The PAM signal can be represented as:

$$s(t)=\sum_n a_n p(t-nT)$$

where a_n represents the amplitude corresponding to each symbol and $p(t)$ is the pulse shape. For binary PAM, the mapping is defined as:

$$a_n=\begin{cases} +A, & \text{for bit } 1 \\ -A, & \text{for bit } 0 \end{cases}$$

At the receiver, demodulation is performed by sampling the received signal and applying a threshold decision rule to recover the original bits.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

num_bits = 20; samples_per_bit = 100; A = 5
bits = np.random.randint(0, 2, num_bits)
symbols = np.where(bits == 1, A, -A)
pam_signal = np.repeat(symbols, samples_per_bit
)t = np.arange(len(pam_signal)) /
samples_per_bit

noise = np.random.normal(0, 1, len(pam_signal))
received_signal = pam_signal + noise
sample_idx = np.arange(samples_per_bit//2, len(received_signal),
    samples_per_bit)
samples = received_signal[sample_idx]
```

```
demod_symbols = np.where(recovered_bits == 1, A, -A)
demod_signal = np.repeat(demod_symbols, samples_per_bit)

plt.figure(figsize=(10
,8)) plt.subplot(4,1,1)
plt.step(range(len(bits)), bits, where='
post') plt.title("Original Bits")
plt.subplot(4,1,2); plt.plot(t,
pam_signal) plt.title("PAM Modulated
Signal") plt.subplot(4,1,3)
plt. plot(t, received_signal )
plt.scatter(sample_idx/samples_per_bit, samples)
plt.title("Received Signal (with Sampling
Points)")
plt. subplot (4 ,1 ,4)
plt.plot(t, demod_signal)
plt.title("Demodulated
Signal") plt.tight_layout ()
plt.show ()
print("Original bits:", bits)
print("Recovered bits:", recovered_bits)
```

Output

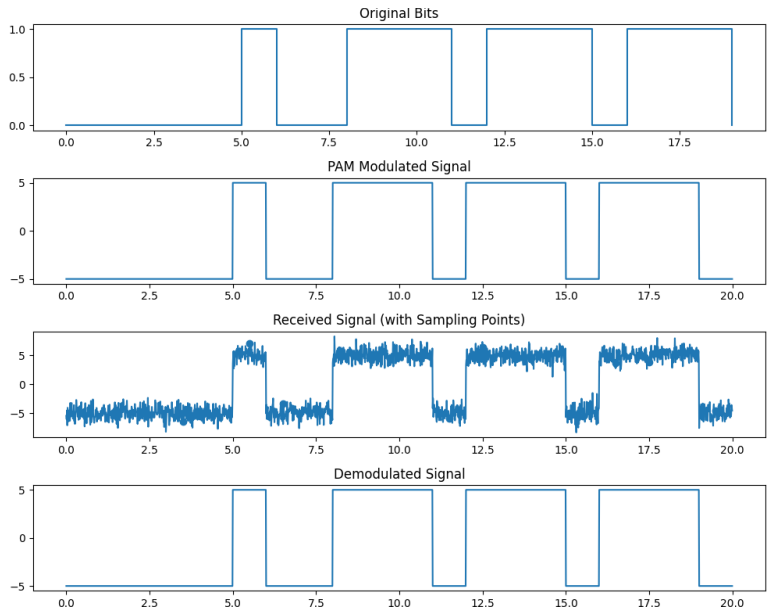


Figure : PAM Modulation and Demodulation with Noise

Results and Discussions

The PAM signal is generated by mapping binary data to amplitude levels. Noise affects the transmitted signal during propagation. Sampling is performed at optimal points for accurate detection. The recovered bits closely match the original transmitted bits. Thus, PAM modulation and demodulation are successfully implemented.

Title

Perform the DPCM modulation and demodulation using MATLAB/ Python.

Objectives

- To understand the concept of Differential Pulse Code Modulation (DPCM).
- To implement DPCM encoding and decoding.
- To analyze prediction and quantization errors.

Theory

Differential Pulse Code Modulation (DPCM) is a signal encoding technique that transmits the difference between successive samples instead of the actual signal values. This reduces redundancy and improves efficiency.

The prediction error is given by:

$$e(n) = x(n) - \hat{x}(n)$$

where $x(n)$ is the original signal and $\hat{x}(n)$ is the predicted signal.

The quantized error is expressed as:

$$e_q(n) = Q[e(n)]$$

At the receiver, the reconstructed signal is obtained by:

$$x_r(n) = x_r(n-1) + e_q(n)$$

Thus, DPCM uses prediction and quantization to efficiently encode the signal.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 1, 200)
x = np.sin(2 * np.pi * 5 * t)

step_size = 0.2
predicted = np.zeros_like(x)
error = np.zeros_like(x)
q_error = np.zeros_like(x)

for i in range(1, len(
    x)): predicted[i] =
    x[i-1]
    error[i] = x[i] - predicted[i]
```

```
reconstructed = np. zeros_like ( x)
for i in range (1 , len ( x)):
    reconstructed [ i] = reconstructed [i-1] + q_error[ i]

plt. figure ( figsize =(10 ,8))
plt.subplot  (4  ,1  ,1)
plt.plot(t,x)  plt. title ("
Original Signal") plt.
subplot(4  ,1  ,2) plt.
plot(t,error) plt. title ("
Prediction Error")
plt.subplot  (4  ,1  ,3) plt.
plot(t,q_error)  plt. title ("
Quantized Error") plt.
subplot  (4  ,1  ,4) plt.
plot(t, reconstructed )
plt. title (" Reconstructed Signal")
plt. tight_layout ()
plt. show ()
```

Output

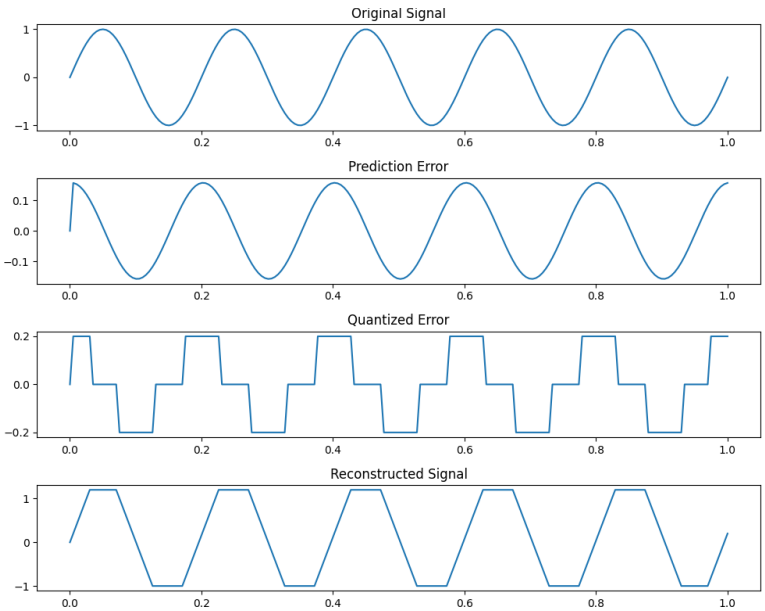


Figure : DPCM Modulation and Demodulation Process

Results and Discussions

The DPCM system encodes the difference between signal samples. Prediction reduces redundancy in the transmitted signal. Quantization introduces small errors in the signal. The reconstructed signal closely follows the original waveform. Thus, DPCM efficiently compresses and reconstructs the signal.